REPORT CARD

# fonsecagabriella/tiny_cat

April 30, 2026

OVERALL SCORE

**5.71** / 7

> *A well-structured, thoughtfully detailed specification that demonstrates strong command of feature planning and testability, with a few consistency gaps and some arithmetic slip-ups holding it back from a top score.*

## Rubric Summary

| Criterion | Score |
|---|---|
| Completeness | 0.88 |
| Clarity and Specificity | 0.87 |
| Internal Consistency (Features) | 0.72 |
| Internal Consistency (Constitution) | 0.72 |
| Testability (Coverage) | 0.88 |
| Testability (Difficulty Levels) | 0.92 |
| Testability (Test Suite) | 0.72 |
| **Total** | **5.71 / 7** |

## Detailed Feedback

## Completeness

0.88

### WHAT WENT WELL

The mission document is a genuine anchor — it explicitly defines the audience, constraints, and ten enumerated success criteria, and every phase ties back to it with traceable IDs. Edge cases and user flows are mapped with scenario/behaviour pairs rather than vague prose, which is exactly the right instinct.

### AREAS TO IMPROVE

- The 'Tired' state is missing from Phase 1 requirements (REQ-06 lists only seven states) even though it appears later in Phases 3, 6, and 7 — its introduction point and tick-order wiring in `runTick` are never formally specified.
- The rule governing `bellyTicks` incrementing (Happiness = 100 per tick) is implied rather than stated as an explicit counter rule, unlike the comparable `hungryTicks` definition.

### AI SPEC-WRITING TIP

Ask your AI to do a "state inventory check" — have it list every game state that appears anywhere in your docs and flag any state that is missing from the Phase 1 state machine definition. This catches omissions like 'Tired' before they cascade across phases.

---

## Clarity and Specificity

0.87

### WHAT WENT WELL

The numeric precision here is genuinely impressive — decay rates, threshold values, penalty tiers, and the full `runTick` execution order are all spelled out with exact numbers and sequence. Lifecycle timings (300 ms poo animation, 1.5 s tap-mark hide) leave no room for guesswork during implementation.

### AREAS TO IMPROVE

- The feed result codes (`'ok'`, `'poo'`, `'full'`, `'invalid'`) are introduced only in the Phase 7 audit rather than being defined upfront in the Phase 1 or Phase 2 requirements where the feed mechanic is first described.

### AI SPEC-WRITING TIP

After drafting each feature's requirements, prompt your AI to list every return value, error code, and result enum that the feature implies — then paste those definitions back into the earliest phase where the mechanic appears.

---

## Internal Consistency (Features)

0.72

**WHAT WENT WELL**

The core mechanics — decay rates, poo thresholds, sick conditions, action effects — stay consistent across all seven feature docs, and the Easter egg trust/willing conditions are cleanly aligned between the feature plan, requirements, and validation for Phase 5.

**AREAS TO IMPROVE**

- Phase 1's feature plan and requirements omit 'Tired' from the state machine priority order entirely, while Phase 7 treats Tired as having been introduced in Phase 3 — the specs never resolve when or whether Tired belongs in Phase 1 logic.
- Phase 7 validation Flow 3 expects 87 total tests while the arithmetic in the same document's task group 2 produces 90 — a direct numerical contradiction within a single feature's docs.

**AI SPEC-WRITING TIP**

Use your AI as a cross-phase consistency checker: paste all your validation doc test-count claims together and ask it to add them up independently. It will catch arithmetic contradictions instantly and flag the discrepancy before you submit.

---

## Internal Consistency (Constitution)

0.72

**WHAT WENT WELL**

The foundational constitution — 30-second tick interval, stat decay rates for Normal and Evolved cats, dark mode default, colour picker default, and localStorage exclusion — is faithfully echoed across all feature docs, which shows disciplined top-down spec writing.

**AREAS TO IMPROVE**

- Phase 1 requirements (REQ-06) and feature plan list only seven states, omitting 'Tired' entirely, which directly contradicts `mission.md` where all eight states including Tired are explicitly defined.
- Phase 3 validation places 'Tired' between Happy and Bored in the priority order, but Phase 1 requirements omit it completely — two feature phases disagree on the same priority list.

**AI SPEC-WRITING TIP**

Maintain a single "canonical state table" in your mission doc and ask your AI to regenerate the priority-order list in every phase doc from that single source, rather than retyping it each time — retyping is where omissions creep in.

---

## Testability (Coverage)

0.88

**WHAT WENT WELL**

The validation docs for Phases 1, 3, 4, 5, and 7 map tightly to their corresponding requirements IDs — decay rates, state expressions, sick suppression of belly events, and Easter egg blocking conditions all have explicit test scenarios that you can trace back to a named requirement.

**AREAS TO IMPROVE**

- Phase 6 validation lacks an explicit test confirming that the help overlay's riddle content matches the exact text specified in mission.md (REQ-P06.3).
- Some state-machine priority-override combinations — such as Belly > Hungry and Belly > Tired — are mentioned in the feature plan but never appear in any validation doc.

**AI SPEC-WRITING TIP**

After writing each validation doc, paste in the corresponding requirements list and ask your AI to identify every requirement that has no matching test scenario — it will surface coverage holes like the missing priority-override combinations in seconds.

---

## Testability (Difficulty Levels)

0.92

**WHAT WENT WELL**

The two-tier validation structure (Level 1 automated/smoke, Level 2 manual/flow) is established clearly in Phase 1 and maintained consistently through all seven phases, with Phase 7 adding a third CI verification layer. This scaffolding makes it easy for a reviewer to know exactly how much effort a given check requires.

**AREAS TO IMPROVE**

- In Phases 4 and 5, one or two sub-flows omit the explicit "Level 1 / Level 2" heading, making the difficulty boundary slightly implicit rather than formally labeled at those points.

**AI SPEC-WRITING TIP**

Ask your AI to apply a validation doc template — with explicit Level 1 / Level 2 headings pre-filled — to every new phase before you start writing the test cases. Structural consistency is the kind of thing a template enforces better than memory.

---

## Testability (Test Suite)

0.72

**WHAT WENT WELL**

Phase 1 specifies a 56-test Jest suite with named files, and Phase 7 audits the gaps, prescribes exactly six new tests with precise setups and assertions, and ties everything to a target count and CI integration. The intention to make the suite verifiable is clearly there.

**AREAS TO IMPROVE**

- `tests/cat.test.js` is specified in table form but no actual `describe`/`it`/`expect` blocks are provided, so the correctness of the suite cannot be verified from the submission alone.
- The final test-count arithmetic is inconsistent across sections — 81 existing tests stated in requirements versus 56 + 24 = 80 from Phase 1, and 87 versus 90 claimed in different parts of Phase 7.

**AI SPEC-WRITING TIP**

Have your AI generate the actual Jest `describe`/`it`/`expect` stubs directly from your table-form test specifications — even skeleton code makes the suite independently verifiable and forces the test count arithmetic to resolve to a single concrete number.

---

## Wrap-up

*This is a genuinely strong submission — the level of numeric precision, the consistent use of requirement IDs, and the multi-tier validation strategy all reflect real spec-writing maturity. Tighten up cross-phase state definitions and let your AI do the arithmetic checks, and you'll have a near-complete professional-grade spec.*